

HW3: Implementing Parallel Matrix Multiplication using CUDA

1. **목표:** CUDA를 이용하여 행렬 곱셈(GEMM)을 parallel로 구현하고 lora 알고리즘 최적화를 한다.

2. Hardware Specification

CUDA Programming에서 Code의 속도의 성능을 향상시키기 위해서는 Hardware specification을 잘 알고, Hardware를 잘 활용할 수 있도록 Code를 작성하여야 한다. 다음은, GPU Programming에서 속도 향상을 위해 어떤 것들을 고려하여야 하는지 살펴본다.

1) My Code Thread Number per SM / Maximum Thread number per SM

2) My Code Shared Memory Usage per Block / Maximum Shared Memory SIZE per Block

3) My Code Register Usage per Block / Maximum Register File Size per Block

Physical Limits for GPU Compute Capability:	8.6
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	16
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	65536
Max Shared Memory per Block	65536
Register allocation unit size	256
Register allocation granularity	warp
Shared Memory allocation unit size	128
Warp allocation granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    1. __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    2. __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

    3. int bx = blockIdx.x; int by = blockIdx.y;
    4. int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
    5. int Row = by * TILE_WIDTH + ty;
    6. int Col = bx * TILE_WIDTH + tx;
    7. float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    8. for (int m = 0; m < Width/TILE_WIDTH; ++m) {
        // Load shared memory phase
        9. subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
        10. subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
        __syncthreads(); // Wait for the copy to finish

        // Compute phase
        12. for (int k = 0; k < TILE_WIDTH; ++k)
        13.     Pvalue += subTileM[ty][k] * subTileN[k][tx];
        14. __syncthreads(); // Wait for the compute to be complete
    15. }
    16. P[Row*Width+Col] = Pvalue;
}
```

위는 Server GPU의 Spec. 및 일반적인 Matrix Multiplication의 Code이다.

Matrix Multiplication의 경우에는 threadIdx.x와 threadIdx.y로 Tile내의 (x,y)좌표를 계산하기 때문에,
ex. `subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH + tx]`

Tile내에서 각 element의 좌표를 threadIdx를 통하여 모두 지정하려면, Tile의 크기 (= TILE_WIDTH * TILE_WIDTH)와 Thread Block의 threads 개수가 같아야 한다.

이는, ThreadBlock에 할당하는 Thread 숫자와 Shared Memory SIZE가 연관된다는 뜻이다. 이를 고려하여, GPU spec에 최적화된 ThreadBlock 당 Threads수를 알아본다.

더불어, Compile Option을 출력하면 다음과 같이 출력된다. 현재 code의 경우 Thread 당 Registers = 40임을 알 수 있다. 따라서, 일단 Thread 당 Registers = 40으로 생각하고, GPU occupancy를 계산한다.

[물론, 완성된 code를 compile한 결과이기에 code에 따라 Register Usage는 다를 수 있다.]

```
$ make build
mkdir -p build
/usr/local/cuda/bin/nvcc -std=c++11 -Xptxas -v -arch=sm_86 -o build/main main.cu
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z10gemm_tiledILi16ELi8EEvPKfS1_Pfiii' for 'sm_86'
ptxas info      : Function properties for _Z10gemm_tiledILi16ELi8EEvPKfS1_Pfiii
                    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 40 registers, 1088 bytes smem, 388 bytes cmem[0]
ptxas info      : Compiling entry function '_Z10gemm_tiledILi8ELi16EEvPKfS1_Pfiii' for 'sm_86'
ptxas info      : Function properties for _Z10gemm_tiledILi8ELi16EEvPKfS1_Pfiii
                    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 40 registers, 1056 bytes smem, 388 bytes cmem[0]
ptxas info      : Compiling entry function '_Z10gemm_tiledILi16ELi16EEvPKfS1_Pfiii' for 'sm_86'
ptxas info      : Function properties for _Z10gemm_tiledILi16ELi16EEvPKfS1_Pfiii
                    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 40 registers, 2112 bytes smem, 388 bytes cmem[0]
ptxas info      : Compiling entry function '_Z12scale_kernelPKfS0_Pffi' for 'sm_86'
ptxas info      : Function properties for _Z12scale_kernelPKfS0_Pffi
                    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 12 registers, 384 bytes cmem[0]
$ |
```

Thread Block Size = 64, 128, 256, 512, 1024의 경우를 비교해본다. (Max Threads per Block = 1024)

	64 threads per block	128 threads per block	256 threads per block	512 threads per block	1024 threads per block
Threads per Block	64	128	256	512	1024
Warps per Block	2	4	8	16	32
Registers per Block	2560	5120	10240	20480	40960
Shared Memory per Block	512 B	1024 B	2048 B	4096 B	8192 B
제약① Warp 수	$48/2 = 24$	$48/4 = 12$	$48/8 = 6$	$48/16 = 3$	$48/32 = 1$
제약② Registers	$65536/2560 = 25$	$65536/5120 = 12$	$65536/10240 = 6$	$65536/20480 = 3$	$65536/40960 = 1$
제약③ Shared Memory	$65536/512 = 128$	$65536/1024 = 64$	$65536/2048 = 32$	$65536/4096 = 16$	$65536/8192 = 8$
제약④ Max Blocks/SM	16	16	16	16	16
Blocks 수/SM	$\min(24, 25, 128, 16) = 16$	$\min(12, 12, 128, 16) = 12$	$\min(6, 6, 32, 16) = 6$	$\min(3, 3, 32, 16) = 3$	$\min(1, 1, 32, 16) = 1$
Warp 수/SM	$16 \times 2 = 32$	$12 \times 4 = 48$	$8 \times 6 = 48$	$16 \times 3 = 48$	$32 \times 1 = 32$
Occupancy	$32/48 = 66\%$	$48/48 = 100\%$	$48/48 = 100\%$	$48/48 = 100\%$	$32/48 = 66\%$
Threads 수/SM	1024	1536	1536	1536	1024

3. How did you implement your parallel algorithm

이번 과제의 목표는 Lora연산을 GPU Programming을 통해 속도 최적화를 하는 것이다. Lora Algorithm은 Matrix Multiplication(GEMM)과 element-wise Scaling & addition으로 나뉜다.

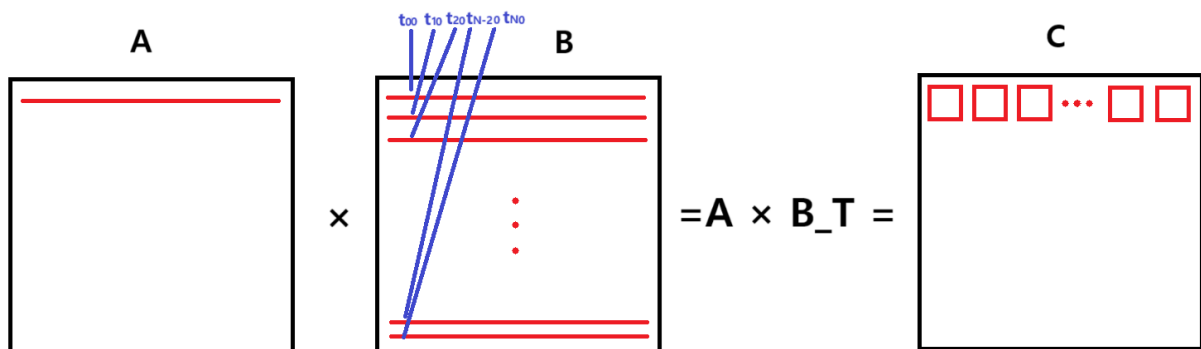
(1) Tiled based Matrix Multiplication Kernel

Input A 와 B 를 받아서 $A \times B^T = C$ 행렬 곱셈 연산을 수행하는 Kernel로 TILE SIZE = TILE_ROW $\times$ TILE_COL로 설정 후 template을 이용하여 Tile 크기를 lora function에서 조절할 수 있도록 하였다.

1.1) Using shared memory

Matrix A 와 B 의 TILE을 Shared Memory에 저장하여 GPU의 Memory Access time을 줄이고, Data Usage의 temporal locality를 높였다. 더불어, Bank Conflict 문제를 완화하기 위하여 threads가 1번에 같은 address의 다른 data를 접근하는 B 의 Shared Memory에는 추가적으로 1개의 Padding을 더하여[(TILE_ROW) $\times$ (TILE_COL+1)]로 설정하였다.

[Matrix Multiplication의 경우 B Matrix는 A Matrix에 비해 서로 다른 Address로 접근이 많음]



1.2) Moving Element of A & B to each shared memory

이번 과제에서의 B Matrix는 transpose를 고려한다. 따라서, Matrix Multiplication을 진행하는 방식은 2가지 선택이 있다.

1.2.1) Matrix B 를 다른 kernel에서 transpose -> B^T 를 받아 Matrix Multiplication

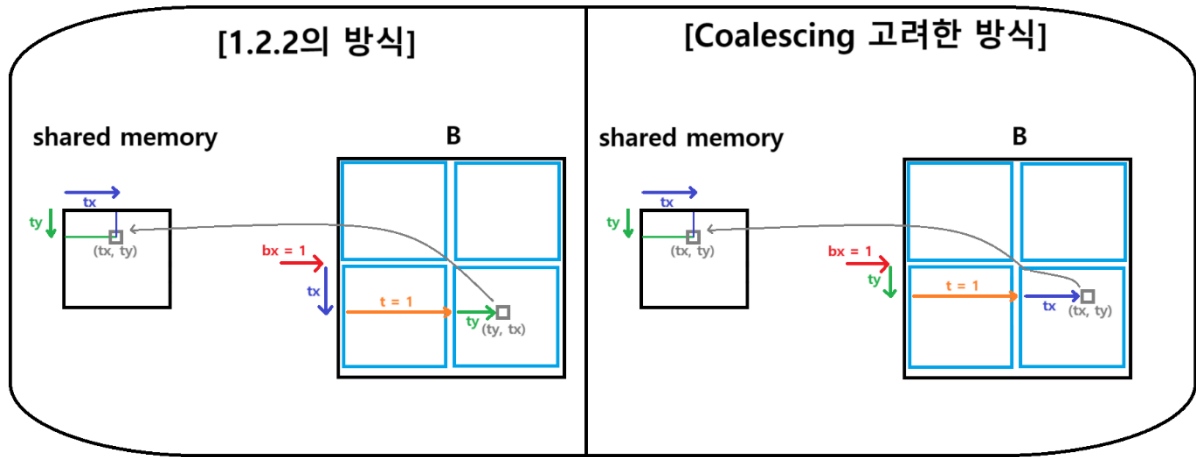
1.2.2) Matrix B 를 shared Memory로 옮길 때 indexing을 이용하여 transpose 후 -> Multiplication

이번 과제에서 내가 선택한 방법은 1.2.2이고, 1.2.1을 선택하지 않은 이유는 transpose를 하기 위해 또다른 kernel을 launching하면 매우 많은 Global Memory Access가 일어나 상당한 Latency가 발생할 것이라고 예상하였기 때문이다. 아래는 1.2.2를 구현한 방식이다. Transpose를 고려하여, tile의 이동방향은 아래쪽이 아닌 오른쪽으로 이동해야 한다.

```
1. for (int t = 0; t < K; t += TILE) {
2.     int b_row = blockIdx.x * TILE + threadIdx.x;
3.     int b_col = t + threadIdx.y;
4.     Bs[threadIdx.y][threadIdx.x] = (b_row < N && b_col < K) ? B[b_row * K + b_col] : 0.0f;
5. }
```

1.3) Considering Coalescing for moving elements to shared memory

위의 1.2.2를 구현한 code를 보면, `int b_col = t + threadIdx.y;`로 coalescing이 일어나지 않고 있다. 1.2.2의 방식을 그림으로 나타내면 아래와 같다.



이 non-coalescing으로 인한 성능 저하를 해결하기 위해 위 그림의 오른쪽 같은 방법을 적용한다. code로는 아래와 같이 구현한다.

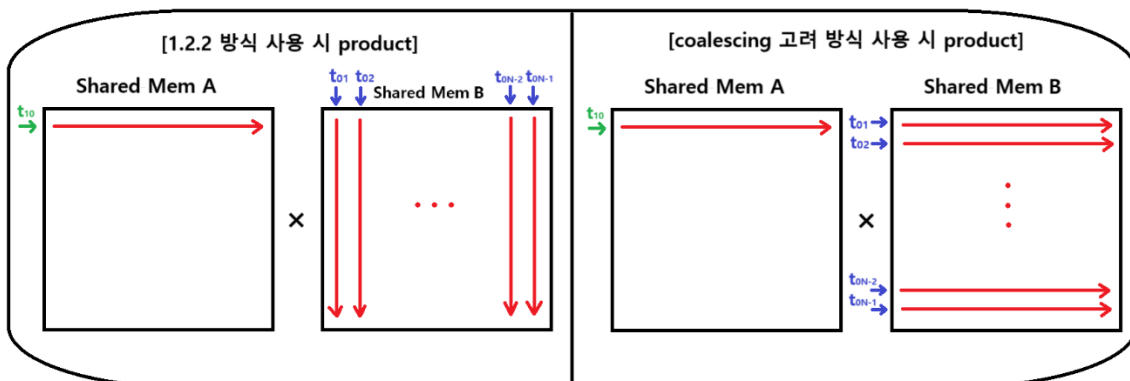
```
1. for (int t = 0; t < K; t += TILE_COL)
2. {
3.     // Load tile from B with transpose consideration
4.     int b_row = block_x * TILE_COL + thread_y;
5.     int b_col = t + thread_x;
6.
7.     Bs[thread_y][thread_x] = B[b_row * K + b_col];
8. }
```

이로써, shared memory로 element를 load하는 과정에서 temporal 및 spatial locality를 모두 최적화하였다.

1.4) computing $A \times B^T$

Coalescing을 고려 시 shared memory로 element를 load하는 과정은 위 그림과 같이 좌표가 transpose되지 않고(= [tx, ty]) 진행되기에, transpose를 고려하여 product 연산을 해야 한다.

즉, 아래 그림과 같이 연산이 되도록 code를 작성해야 한다.



여기서 주의할 점은 shared memory B의 element를 load할 때, row는 t_y 를 이용하여 계산했었기에 관리하기에 row를 t_y 를 이용하여 아래 코드와 같이 code를 작성하면 문제점이 발생한다는 것이다.

```
1. for (int k = 0; k < TILE_COL; ++k)
2.     acc += As[thread_y][k] * Bs[thread_y][k];
```

왜냐하면, As(= shared Memory A)의 1개의 row는 Bs(= shared Memory B)의 모든 row와 연산이 되어야 하는데, 위 code는 As의 1개의 row가 Bs의 As와 같은 row에 있는 1개의 row와만 연산되기 때문이다.

따라서, Bs의 row는 t_x 를 이용하여 계산해야 한다. 구현은 아래와 같이 한다.

```
1. for (int k = 0; k < TILE_COL; ++k)
2.     acc += As[thread_y][k] * Bs[thread_x][k];
```

이 경우 t_x 를 이용하여 Bs를 접근하기 때문에, As의 t_y 가 1개의 row를 접근하는 동안 Bs의 모든 row와 연산이 된다.

[참고] t_y 와 t_x 의 동작이 다른 이유는 thread global idx가 아래와 같이 계산되기 때문이다.

```
Thread_global_idx = blockIdx.x * blockDim.x * blockDim.y + threadIdx.y * blockDim.x + threadIdx.x;
```

즉, 하나의 t_y 마다 t_x 가 blockDim.x만큼 존재한다.

1.5) considering non-square tiling

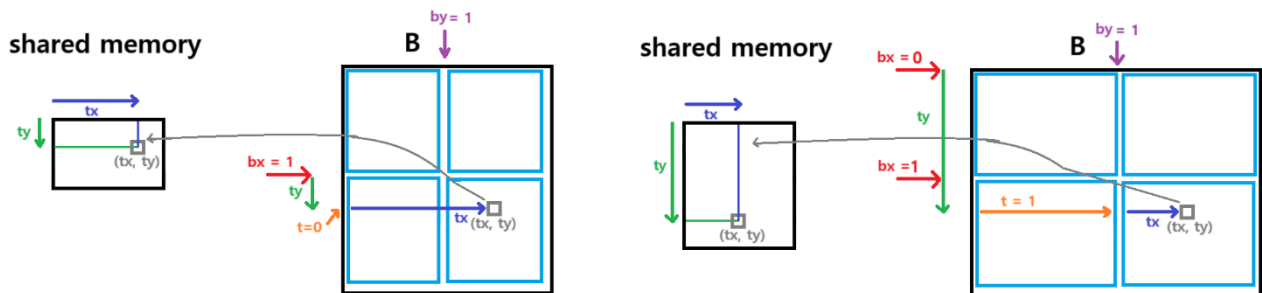
2번에서 분석한, Hardware specifications를 고려하여 최적의 성능을 내기 위해서는 non-square tiling 또한 고려해야 한다.

non-square tiling의 경우 A와 B의 elements를 shared memory로 옮기는 과정과 Matrix Multiplication을 연산하는 과정이 square tile과 어떻게 다른지 고려해줘야 한다.

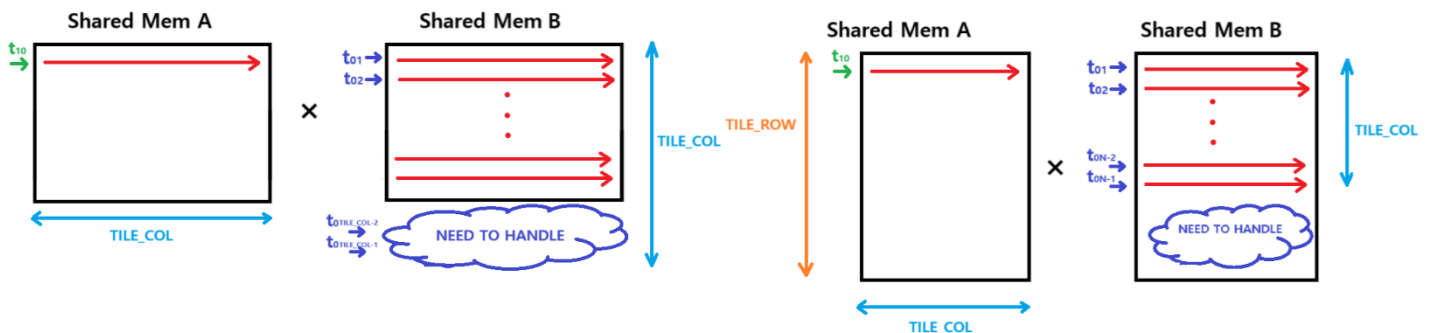
Global memory의 elements를 Shared memory로 옮기는 과정에서는 TILE_WIDTH를 서로 다르게 하면, 1.3)에서 다루었던 방식대로 load를 하면 Redundancy가 생기는 문제가 발생한다. 아래 그림을 보면, $TILE_ROW < TILE_COL$ 의 경우 tx가 by의 경계를 넘어버리고, 반대의 경우 ty가 bx의 경계를 넘어버린다. 또한 TILE의 shape와 Matrix B를 구분하는 block shape가 같지 않은 문제가 발생한다. (그림에서 shared memory[검정 테두리]와 block[하늘색 테두리]의 shape가 다름)

또한, 1.4)에서 `acc += As[thread_y][k] * Bs[thread_x][k];`와 같이 Matrix Multiplication을 수행하는 부분에서는 Redundant한 부분을 Compute하여 같은 부분을 2번 계산하거나, 배열의 range를 넘어가 runtime error가 발생한다.

[Problem with loading Phase]



[Problem with Computing Phase]



그렇다면, bx와 by 및 tx와 ty를 Matrix B에 맞춰서 다시 정의해주면 될 것이라고 예상할 수 있다. 하지만, 나는 실패하였다. 실패한 이유를 설명한다.

-- 시도한 방법 --

1.5.1) 먼저, 2D Block 및 2D Grid로 선언 시 bx와 by 및 tx와 ty를 새로운 축으로 전환하는 계산과정이 1D로 선언하였을 때보다 더 복잡하므로 1D Block 및 Grid로 code를 변환하였다.

1.5.2) 아래를 선언하여, transpose된 shape을 가진 block을 활용한다.

```
1. // Map 1D thread index to 2D tile coordinates for transpose
2. const int thread_y_T = tid / TILE_ROW;
3. const int thread_x_T = tid % TILE_ROW;
4. // 1D block index to compute row and col of C considering transpose
5. const int block_y_T = block_id / ((N + TILE_ROW - 1) / TILE_ROW);
6. const int block_x_T = block_id % ((N + TILE_ROW - 1) / TILE_ROW);
```

1.5.3) Load하는 과정은 아래와 같이 처리하였다. (bx를 ty의 크기에 맞추기 위하여)

```
1. // Load tile from B and tile should handle B_T
2. int b_row = block_x_T * TILE_COL + thread_y;
3. int b_col = t + thread_x;
4. Bs[thread_y * (TILE_COL + 1) + thread_x] = (b_row < N && b_col < K) ? B[b_row * K + b_col] : 0.0f;
```

1.5.4) compute하는 과정은 아래와 같이 처리하였다. (tx의 크기를 맞추기 위하여)

```
1. for (int k = 0; k < TILE_COL; ++k)
2.     acc += As[thread_y * TILE_COL + k] * Bs[thread_x_T * (TILE_COL + 1) + k];
```

문제점은 예시를 들어 설명하겠다. TILE_ROW = 8, TILE_COL = 32라고 하자.

thread_y = 0 ~ 7까지의 범위를 가지고, thread_y 값 하나 당 thread_x는 0 ~ 31의 값을 가진다.

thread_y_T의 경우 0~31까지의 범위를 가지고, thread_y_T 값 하나당 thread_x_T는 0 ~ 7의 값을 가진다.

block_x의 경우 block 당 8×32 threads를 가지고, block_y는 block_x가 N/TILE_COL 일 때마다 1개이다.

block_x_T의 경우 block 당 8×32 threads를 가지고, block_y_T는 block_x_T가 N/TILE_ROW 일 때마다 1개이다.

Non-square tiling을 구현하는 과정에 있어서 발생한 문제점은 thread idx간 synchronization을 고려하지 않았다는 점이다. 즉, As로 load하는 A Matrix는 block_y를 기준으로

$A[(\text{block_y} * \text{TILE_ROW} + \text{thread_y}) * K + t + \text{thread_x}]$ 가 load되고,

Bs로 load하는 B Matrix는 block_x_T를 기준으로

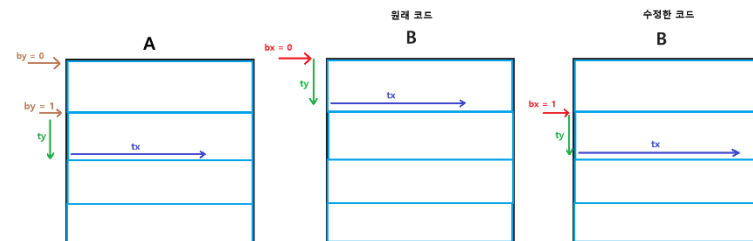
$\text{b_row} = \text{block_x_T} * \text{TILE_COL} + \text{thread_y};$ 가 load된다.

하지만, block_y는 TILE_COL을 기준으로 증가하고, block_x_T는 TILE_ROW를 기준으로 증가하기에, data를 shared memory로 load하는 과정에서 각 shared memory간의 idx가 뒤틀린다. 더불어, thread_y와 block_x_T 또한, idx가 증가하는 방식이 다르고, synchronized되어 있지 않기에, 잘못된 data를 load한다.

예를 들어, 32×32 Matrix A, 32×32 Matrix B가 존재하고, 이를 8×32 TILE로 TILING 할 시 어떤 문제가 생기는 지 살펴본다.

$$\text{globalIdx} = \text{blockIdx.x} \times \text{blockDim.x} \times \text{blockDim.y} + \text{threadIdx.x} \times \text{blockDim.x} + \text{threadIdx.y}$$

항목	원래 코드	수정한 코드
globalIdx	504	504
threadIdx.x	$(504 \% 256) \% 32 = 24$	$(504 \% 256) \% 8 = 0$
threadIdx.y	$(504 \% 256) / 32 = 7$	$(504 \% 256) / 8 = 31$
blockIdx.x	$(504 / 256) \% (32 / 32) = 0$	$(504 / 256) \% (32 / 8) = 1$
blockIdx.y	$(504 / 256) / (32 / 32) = 1$	$(504 / 256) / (32 / 8) = 0$



제대로 된 transpose Matrix연산이 되려면 A의 1번째 TILE의 각각의 row는 B의 1개 TILE의 row와 모두 load되어 연산되며, B의 다음 TILE(=bx: 0->1)의 row와 모두 load되어 연산되고, 이를 반복하여 A의 1번째 TILE의 각각의 row들은 B의 모든 row와 연산된다. A의 나머지 TILE도 마찬가지이다.

하지만, 위의 표와 그림을 보면 알 수 있는 것은 수정된 코드는 제대로 된 연산의 진행 방향과는 상당히 다르게 진행된다는 것이다. 위 예시에서 Matrix B의 bx = 0일 때는 Matrix A의 by = 0인 TILE이 load되고, Matrix B의 bx = 1일 때는 Matrix A의 by = 1인 TILE이 load된다. 즉, Matrix B의 bx = 1일 때, Matrix A의 by = 0인 TILE이 load되지 않아, 연산에 오류가 생기는 것이다.

결국, 모든 index가 꼬여버린 것이다.

Thread (224, 0) - A[7][k] x B[0][k] = C[31][0]: 0.000000	Thread (240, 0) - A[15][k] x B[0][k] = C[31][0]: -29.059700
Thread (0, 0) - A[0][k] x B[0][k] = C[0][0]: 51.899090	Thread (0, 0) - A[0][k] x B[0][k] = C[0][0]: 51.899090
Thread (225, 0) - A[7][k] x B[1][k] = C[31][1]: 0.000000	Thread (241, 0) - A[15][k] x B[1][k] = C[31][1]: 17.294596
Thread (1, 0) - A[0][k] x B[1][k] = C[0][1]: 51.418636	Thread (1, 0) - A[0][k] x B[1][k] = C[0][1]: 51.418636
Thread (226, 0) - A[7][k] x B[2][k] = C[31][2]: 0.000000	Thread (242, 0) - A[15][k] x B[2][k] = C[31][2]: 18.793930
Thread (2, 0) - A[0][k] x B[2][k] = C[0][2]: 75.494690	Thread (2, 0) - A[0][k] x B[2][k] = C[0][2]: 75.494690
Thread (3, 0) - A[0][k] x B[3][k] = C[0][3]: 56.906353	Thread (3, 0) - A[0][k] x B[3][k] = C[0][3]: 56.906353
Thread (227, 0) - A[7][k] x B[3][k] = C[31][3]: 0.000000	Thread (243, 0) - A[15][k] x B[3][k] = C[31][3]: 30.006470
Thread (4, 0) - A[0][k] x B[4][k] = C[0][4]: 94.098083	Thread (4, 0) - A[0][k] x B[4][k] = C[0][4]: 94.098083
Thread (228, 0) - A[7][k] x B[4][k] = C[31][4]: 0.000000	Thread (244, 0) - A[15][k] x B[4][k] = C[31][4]: 23.703918
Thread (229, 0) - A[7][k] x B[5][k] = C[31][5]: 0.000000	Thread (5, 0) - A[0][k] x B[5][k] = C[0][5]: 73.411987
Thread (5, 0) - A[0][k] x B[5][k] = C[0][5]: 73.411987	Thread (245, 0) - A[15][k] x B[5][k] = C[31][5]: -57.969521
Thread (230, 0) - A[7][k] x B[6][k] = C[31][6]: 0.000000	Thread (246, 0) - A[15][k] x B[6][k] = C[31][6]: 46.975632
Thread (6, 0) - A[0][k] x B[6][k] = C[0][6]: -83.641060	Thread (6, 0) - A[0][k] x B[6][k] = C[0][6]: -83.641060
Thread (7, 0) - A[0][k] x B[7][k] = C[0][7]: 8.012438	Thread (247, 0) - A[15][k] x B[7][k] = C[31][7]: -21.581696
Thread (231, 0) - A[7][k] x B[7][k] = C[31][7]: 0.000000	Thread (7, 0) - A[0][k] x B[7][k] = C[0][7]: 8.012438
Thread (8, 0) - A[0][k] x B[8][k] = C[0][8]: 51.899090	Thread (248, 0) - A[15][k] x B[8][k] = C[31][8]: 0.659815
잘못된 Indexing으로 인한 결과	올바른 결과
	Thread (8, 0) - A[0][k] x B[8][k] = C[0][8]: 8.768517

위 그림은 잘못된 indexing으로 인한 잘못된 결과값이 나오는 것을 보여준다. 처음에는 모두 0부터 시작하기에, 제대로 된 결과가 나오는 것을 볼 수 있지만, C의 31번째 row의 data들 및 C[0][8]의 결과를 보면, idx가 뒤틀려서 잘못된 결과가 나오는 것을 볼 수 있다.

Idx를 해결하려면, TILE_ROW와 TILE_COL에 대한 for loop를 따로따로 만들어준 후 shared memory에 load를 해야 할 것 같다고 예상된다. 더불어, 추가적인 for loop 및 연산을 하는 것은 오히려 thread 당 연산 횟수 및 branch Inst에 대한 부담이 증가하기에 non-square tiling을 사용 시 성능이 오히려 안 좋아질 것으로 예상된다.

(2) Scaling and Sum kernel

Input A 와 B 를 받아서 $A + \text{scale} * B = C$ 행렬 scalar 곱셈 및 덧셈 연산을 수행하는 Kernel이다.

이 경우에는 thread마다 A 와 B 의 index를 담당하여, 곱셈 및 덧셈 연산을 수행하면 되기에, global idx를 계산 후 단순 곱셈 및 덧셈을 수행하였다.

```
1. int i = blockIdx.x * blockDim.x + threadIdx.x;
2. y[i] = a[i] + scale * b[i];
```

(3) LoRA 함수

LoRA함수는 아래의 구문을 연산하여 최종 output인 $\text{out_linear} + \text{out_lora}$ 를 얻는 함수이다.

```
1. out_linear = x @ W.T # [B, out_dim]
2. out_lora = x @ A.T @ B.T * scale # [B, out_dim]
3. return out_linear + out_lora
```

이번 과제에서는 CUDA를 이용한 LoRA 연산을 진행하기에, lora함수는 위에서 살펴본 kernel을 불러오고, 중간 결과인 $x @ W.T$ 와 $x @ A.T$ 를 임시로 저장해 놓을 temporary buffer를 allocate하는 과정을 수행한다.

Kernel의 실행 과정은 다음과 같다.

1번째 Kernel: $x @ W.T$ 를 수행하여 d_linear 라는 buffer에 저장

2번째 Kernel: $x @ A.T$ 를 수행하여 d_tmp 라는 buffer에 저장

3번째 Kernel: d_tmp 를 이용하여 $x @ A.T @ B.T$ 계산 및 d_linear 를 이용하여 scale & sum 진행

4. Reasons why your program is performing at its best and proving whether the performance margin is there or not

(1) Occupancy 최적화

GPU 성능에 매우 큰 영향을 주는 Occupancy는 GPU의 Multiprocessor(SM)에서 동시에 실행되는 warp의 비율로 결정된다. 2.에서 알아본 바에 의하면 Thread Block의 크기를 128, 256, 512로 설정함으로써 Occupancy를 100%로 유지할 수 있다.

사실, 위에서 제대로 된 non-square tiling을 이용한 transposed matrix multiplication을 구현하지는 못했지만, redundant한 연산이 생길 뿐, TILE의 크기를 non-square로 정의하여 Kernel Launch를 할 수 있다. 즉, Occupancy를 100%로 하고, redundant한 연산을 하는 것 또는 Occupancy를 67%정도로 하는 대신, 필요한 연산만 하는 것에 대해 고민을 해봐야 했다.

Occupancy를 100%로 하기 위해 non-square tile로 kernel을 launch하는 장점은 occupancy가 100%

이므로 모든 thread가 동작하는 이점 및 실행되는 Warp의 개수가 많아져, Latency Hiding이 잘 된다는 점이다.

직접 실행시간을 측정하여 비교한 결과 occupancy를 100%로 유지하고, redundant한 연산을 하는 것이 속도가 근소하게 빨랐다. 따라서, non-square tile로 kernel을 launch하였다.

(2) Shared Memory 활용

Global Memory 접근의 latency 문제를 해결하고 temporal locality를 향상시키기 위해서는 Shared Memory를 활용하여야 한다. 즉, TILE 단위로 data를 cache가 아닌 Shared Memory에 저장하여, cache였으면 eviction이 일어날 수 있는 요소를 shared memory를 이용하여 eviction이 일어나지 않도록 하였고, 결과적으로 data reuse가 되며, temporal locality가 높아지는 효과이다. 특히, Shared Memory는 SRAM으로 설계되고, 더 가까운 거리의 빠른 통신을 이용하기 때문에, DRAM으로 설계되고, 비교적 느린 통신을 이용하는 Global Memory보다 더 빠른 Access time을 가진다.

(3) Bank Conflict 및 Coalescing 고려

Shared memory에서의 Bank Conflict 및 Global Memory의 elements를 Shared Memory로 load하거나 다시 Global Memory로 write할 때의 Coalescing 문제를 모두 해결하였다.

Matrix B에서는 앞에서 언급하였듯이 같은 Port(또는 Address)에 여러 threads가 서로 다른 Data들을 많이 접근하는데, 이는 Bank Conflict의 주요 원인이다. 따라서, Shared Memory에 padding을 추가함으로써 Bank Conflict 현상을 최소화하였다.

3.에서 설명했듯이 thread indexing 방식을 수정하여 data load 및 연산 후 data store시 coalesced memory 접근을 수행할 수 있도록 하였다. Coalescing 접근을 할 시, 연속된 Threads(= tx가 연속)가 연속되 Global Memory 주소를 접근하여, 중간과정에서 data load의 매개체 역할을 할 cache(cf. cache는 programmer에게 transparent하기에 code에서 직접적으로 cache 주소를 지정해 줄 수는 없다)의 cache hit rate를 높여 spatial locality가 향상된다.

(4) Kernel 호출 최소화

Matrix Multiplication을 하기 이전에 미리, Matrix를 transpose하는 추가적인 Kernel을 사용하여 Matrix를 미리 transpose하고, 그 이후에 기본적인 Matrix Multiplication을 하는 방법도 존재하였다. 하지만, 추가적인 Transpose Kernel을 Launch할 경우 kernel launch time 및 상당히 많은 양의 Memory Access가 Transpose 연산에서 발생하기 때문에, Latency가 상당할 것으로 예상되었다.

따라서, Matrix Multiplication Kernel 자체에서 data loading 및 compute 시 indexing을 수정하여

transpose를 수행함 transpose kernel launch latency overhead를 제거하였다.

(5) Register 및 Shared Memory 사용량 최적화

Register 사용량의 경우 compile시 compile option을 추가하면 사용량을 출력할 수 있었다. 출력된 결과는 thread 당 40 registers로 매우 이상적인 register 사용량이었다.

Shared Memory 사용량의 경우, Learn Us에 올라간 CUDA occupancy Excel파일과 cmd에서 출력한 결과가 달라서, cmd의 출력을 기준으로 code를 작성하였다. (48KB per block)

```
Device name: NVIDIA GeForce RTX 3090
Total global memory: 24252 MB
Max threads per block: 1024
Max threads per SM: 1536
Shared memory per block: 48 KB
Registers per block: 65536
Warp size: 32
Multiprocessor count: 82
```

현재, Max threads per block = 1024로 만약, block SIZE를 1024로 선언하여도, Shared Memory SIZE는 block SIZE에 dependent하므로, $2\text{Memory} \times 4\text{Byte} \times 1024\text{Elements} = 8\text{KB}$ 만큼 사용한다. 따라서, Shared Memory의 SIZE를 초과하지 않았기에 추가적인 overhead는 없을 것으로 예상할 수 있다.

(6) if-else문 최대한 제거

GPU의 경우 if-else와 같은 branch구문에서는 dependency가 생기기 때문에 execution unit이 serial 하게 작동하게 되고, 이는 execution unit의 활용도를 저하한다. 따라서, Kernel안에서는 최대한 if-else문을 제거하는 것이 좋다.

Matrix Multiplication에서 If-else문은 주로 Matrix C의 범위(= $M \times N$)를 넘어가는 연산을 하였을 때, 이 연산결과를 output에 반영하지 않기 위하여 사용한다.



오른쪽 그림과 같이 TILE의 크기가 Matrix 크기가 Align되지 않는 경우를 줄여 if-else문을 사용하지 않아도 올바른 결과가 연산 되도록 하였다.

(7) Loop Unrolling

Tile을 순회하며, data를 shared memory에 load하는 부분과 partial sum을 accumulate하는 부분을 8단위로 Unrolling하여, Branch 조건 검사의 영향을 줄이고자 하였다.

(8) Performance Margin left??

당연히 Performance Margin이 남아있을 것이다. 이번 과제에서 매우 오랜 시간동안 어떻게 해야 transpose kernel을 따로 launch하지 않고, coalesced한 indexing만으로 non-square tiling matrix

multiplication을 구현할 수 있을지 고민하였다. 하지만, redundant한 연산을 생기지 않게 하는 완벽한 non-square tiling transposed matrix multiplication kernel구현에 실패하였다.

따라서, $x @ A.T @ B.T$ 의 연산 결과 중 거의 절반에 해당하는 연산이 버려진다. 따라서, 전체 kernel run time중 $x @ W.T$ 는 강의안에서 배운 방법만 사용하였을 때 최대한 최적화되었고, $x @ A.T @ B.T$ 는 최적화 요소가 남아있다.

$$x @ W.T \Rightarrow x[32][4096] \times W.T[4096][4096] = 32 \times 4096 \times 4096 \text{ computations} = 536,870,912$$

$$x @ A.T \Rightarrow x[32][4096] \times A.T[4096][8] = 32 \times 4096 \times 8 \text{ computations} = 1,048,576$$

$$x @ A.T @ B.T \Rightarrow tmp[32][8] \times B.T[8][4096] = 32 \times 8 \times 4096 \text{ computations} = 1,048,576$$

scaling & sum => almost no overhead with computation

이므로 *by using Amdahl's Law*

$$\text{Can be Optimized portion: } \frac{2,097,152}{538,968,064} = 0.4\%$$

따라서, 버려지는 50%의 연산을 안 낭비되게 만들면 50%의 성능향상을 기대할 수 있다.

$$\text{Expected Performance Gain} = \frac{100}{99.6 + \frac{0.4}{2}} = 0.2\% \text{ Gain}$$

5. Evaluation including tables or graphs

Case A: Shared Mem X / Coalescing X / Block SIZE = 16×16

Case B: Shared Mem O / Coalescing X / Block SIZE = 16×16

Case C: Shared Mem O / Coalescing O / Block SIZE = 16×16 / Has Bank Conflict

Case D: Shared Mem O / Coalescing O / Block SIZE = 16×16 / Prevent Bank Conflict

Case E: Shared Mem O / Coalescing O / 1st kernel launch Block SIZE = 16×16 /

2nd kernel launch Block Size = 8×8 / 3rd kernel launch Block Size = 8×8 /

Prevent Bank Conflict

Case F: Shared Mem O / Coalescing O / 1st kernel launch Block SIZE = 16×16 /

2nd kernel launch Block Size = 8×16 / 3rd kernel launch Block Size = 16×8 /

Prevent Bank Conflict

Run Time	Case A (ms)	Case B (ms)	Case C (ms)	Case D (ms)	Case E (ms)	Case F (ms)
1 trial	2.92	1.053	1.068	0.768	0.744	0.725
2 trial	2.934	1.057	1.206	0.84	0.839	0.751
3 trial	2.967	1.057	1.155	0.812	0.783	0.743
4 trial	2.959	1.057	1.267	0.839	0.861	0.747
5 trial	2.914	1.055	1.17	0.741	0.845	0.896
AVG.	2.939	1.056	1.173	0.792	0.814	0.772

6. Something else

6-1. GPU 0번 & 1번 존재

현재 Server에는 RTX 3090 GPU가 2대 존재한다. 따라서, GPU 2대를 모두 활용할 수 있는 방식으로 Code를 구성하면 Ideallyy 2x performance를 기대할 수 있을 것 같다. 물론, Data Transfer Latency로 인하여 Reality에서는 2x 까지 성능 향상은 바랄 수 없겠지만, 상당히 빠른 속도를 달성할 수 있을 것이라고 예상한다.

6-2. non-square tiling 달성 시 performance gain

$$\text{Expected Performance Gain} = \frac{100}{99.6 + \frac{0.4}{2}} = 0.2\% \text{ Gain}$$

사실 매우 적은 양의 gain이지만, non-square Matrix multiplication을 성공하고 싶었다. 하지만, 많은 시간을 투자했지만, 구현에 실패하여, 자기자신에게 많이 실망하였다.

6-3. Eliminating if-else for storing computed elements to Matrix C

If-else문을 최대한 제거하였지만, 연산된 결과를 Matrix C에 store 시 non-square tiling을 완벽하게 구현하지 못하여 if-else문을 사용하지 않으면, 잘못된 연산결과가 C에 write되기에 if-else문을 넣어 범위에서 벗어나는 결과가 C에 write되는 것을 방지하였다. 만약, C에 write하는 과정에서의 if-else문을 eliminate 할 수 있으면 추가적인 performance gain이 존재할 것이라고 생각한다.

7. Reference

- 4_cuda_mgp_2025.pdf
- 5_cuda_matmul_mgp_2025.pdf
- 6_Transpose_bankConflict_mgp_2025.pdf